ASSIGNMENT- 20.5

R.Abhinav Reddy

2303A52348

Batch: 44

Task 1 – Password Storage Vulnerability Check

Prompt:

Generate a Python user registration and login system that stores user credentials (username and password) in a JSON file. Do not use any encryption or hashing for passwords. Keep the implementation simple.

Code:

```python
import hashlib
import hmac
import json
import os

DB_FILE = "users.json"


def load_users():
    if os.path.exists(DB_FILE):
        with open(DB_FILE, "r", encoding="utf-8") as f:
            return json.load(f)
    return {}


def save_users(users):
    with open(DB_FILE, "w", encoding="utf-8") as f:
        json.dump(users, f, indent=2)


def hash_password(password, salt=None):
    salt = salt or os.urandom(16).hex()
    digest = hashlib.sha256((salt + password).encode()).hexdigest()
    return f"{salt}${digest}"


def verify_password(password, stored):
    salt, saved_digest = stored.split("$", 1)
    new_digest = hash_password(password, salt).split("$", 1)[1]
    return hmac.compare_digest(saved_digest, new_digest)


users = load_users()
print("1. Register")
print("2. Login")
choice = input("Choose (1/2): ").strip()
username = input("Username: ").strip()
password = input("Password: ").strip()
```

```python
if choice == "1":
    if username in users:
        print("User already exists")
    else:
        users[username] = hash_password(password)
        save_users(users)
        print("Registered (password saved as hash)")
elif choice == "2":
    if username in users and verify_password(password, users[username]):
        print("Login successful")
    else:
        print("Invalid username or password")
else:
    print("Invalid option")
```

Output:

```
(.venv) PS D:\AI Assisted coding> & "d:/AI Assisted coding/.venv/Scripts/python.exe" "d:/AI Assisted coding/20.5.py"
1. Register
2. Login
Choose (1/2): 1
Username: Induvardhan1
Password: Induvardhan123
Registered (password saved as hash)
```

```
(.venv) PS D:\AI Assisted coding> & "d:/AI Assisted coding/.venv/Scripts/python.exe" "d:/AI Assisted coding/20.5.py"
1. Register
2. Login
Choose (1/2): 2
Username: Induvardhan1
Password: Induvardhan123
Login successful
(.venv) PS D:\AI Assisted coding>
```

Observation:

The code securely hashes passwords using SHA-256 with a unique salt for each user. The use of hmac.compare_digest prevents timing attacks when verifying passwords. User data is stored in a JSON file, which is simple but not suitable for large applications or production use.

Task 2 – Hardcoded Secrets Detection

Prompt:

Generate a Python script that connects to a public API (like a weather or news API) using an API key. Include the API key directly in the source code and use it to fetch and display data

Code:

```python
import os
from typing import Any
import requests

API_URL = "https://api.github.com/user"

def load_api_key() -> str:
    api_key = os.getenv("GITHUB_TOKEN", "").strip()
    if not api_key:
        raise RuntimeError("Missing GITHUB_TOKEN environment variable.")
    return api_key

def fetch_profile() -> dict[str, Any]:
    token = load_api_key()
    headers = {
        "Authorization": f"Bearer {token}",
        "Accept": "application/vnd.github+json",
    }
    response = requests.get(API_URL, headers=headers, timeout=10)
    response.raise_for_status()
    return response.json()

if __name__ == "__main__":
    try:
        profile = fetch_profile()
        print("Authenticated user:", profile.get("login", "unknown"))
    except Exception as exc:
        print("API call failed:", exc)
```

```
Authenticated user: Induvardhan
```

Task 3 – File Upload Security Validation

Prompt:

Generate a basic Flask web application that allows users to upload files to the server. Do not include any validation for file type, file size, or filename security.

Code:

```python
import os
import uuid
from pathlib import Path

from flask import Flask, jsonify, request
from werkzeug.utils import secure_filename


UPLOAD_DIR = Path("uploads")
UPLOAD_DIR.mkdir(exist_ok=True)

ALLOWED_EXTENSIONS = {"png", "jpg", "jpeg", "pdf", "txt"}
ALLOWED_MIME_TYPES = {
    "image/png",
    "image/jpeg",
    "application/pdf",
    "text/plain",
}
MAX_FILE_SIZE = 2 * 1024 * 1024

app = Flask(__name__)
app.config["MAX_CONTENT_LENGTH"] = MAX_FILE_SIZE


def is_allowed(filename: str, mime_type: str) -> bool:
    if "." not in filename:
        return False
    ext = filename.rsplit(".", 1)[1].lower()
    return ext in ALLOWED_EXTENSIONS and mime_type in ALLOWED_MIME_TYPES


@app.post("/upload")
def upload_file():
    if "file" not in request.files:
        return jsonify({"error": "No file part"}), 400

    file = request.files["file"]
    if file.filename == "":
        return jsonify({"error": "No selected file"}), 400
```

```python
    mime_type = file.mimetype or ""
    if not is_allowed(file.filename, mime_type):
        return jsonify({"error": "File type not allowed"}), 400

    original_name = secure_filename(file.filename)
    ext = original_name.rsplit(".", 1)[1].lower()
    unique_name = f"{uuid.uuid4().hex}.{ext}"
    target_path = UPLOAD_DIR / unique_name
    file.save(target_path)

    return jsonify(
        {
            "message": "Upload successful",
            "saved_as": unique_name,
            "original_name": original_name,
        }
    )


if __name__ == "__main__":
    app.run(debug=False)
```

```
(.venv) PS D:\AI Assisted coding> & "d:/AI Assisted coding/.venv/Scripts/python.exe" "d:/AI Assisted coding/task3_secure_upload.py"
 * Serving Flask app 'task3_secure_upload'
 * Debug mode: off
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
 * Running on http://127.0.0.1:5000
Press CTRL+C to quit
```

Observation:

This code implements a secure file upload endpoint using Flask. It checks for allowed file extensions and MIME types, limits the maximum file size, and uses secure filenames to prevent directory traversal attacks. Uploaded files are saved with unique names to avoid collisions.

Task 4 – Command Injection Prevention

Prompt:

Generate a Python script that takes user input from the command line and executes a system command using os.system() or subprocess. The user input should be directly passed into the command without validation.

Code:

```python
import re
import subprocess


ALLOWED_COMMANDS = {
    "ping": [r"^[a-zA-Z0-9.-]{1,253}$"],
    "nslookup": [r"^[a-zA-Z0-9.-]{1,253}$"],
}


def validate_args(command: str, args: list[str]) -> bool:
    patterns = ALLOWED_COMMANDS.get(command)
    if patterns is None or len(args) != len(patterns):
        return False
    return all(re.fullmatch(pattern, value) for pattern, value in zip(patterns, args))


def run_safe_command(command: str, args: list[str]) -> str:
    if command not in ALLOWED_COMMANDS:
        raise ValueError("Command not allowed.")
    if not validate_args(command, args):
        raise ValueError("Invalid arguments.")

    result = subprocess.run(
        [command, *args],
        capture_output=True,
        text=True,
        check=False,
        timeout=10,
    )
    return result.stdout if result.returncode == 0 else result.stderr


if __name__ == "__main__":
    cmd = input("Command (ping/nslookup): ").strip().lower()
    value = input("Target host/domain: ").strip()
    try:
        output = run_safe_command(cmd, [value])
        print(output)
    except Exception as exc:
        print("Rejected:", exc)
```

```
Command (ping/nslookup): ping
Target host/domain: google.com

Pinging google.com [142.250.182.78] with 32 bytes of data:
Reply from 142.250.182.78: bytes=32 time=14ms TTL=116
Pinging google.com [142.250.182.78] with 32 bytes of data:
Reply from 142.250.182.78: bytes=32 time=14ms TTL=116
Reply from 142.250.182.78: bytes=32 time=14ms TTL=116
Reply from 142.250.182.78: bytes=32 time=13ms TTL=116
Reply from 142.250.182.78: bytes=32 time=10ms TTL=116
Reply from 142.250.182.78: bytes=32 time=12ms TTL=116

Ping statistics for 142.250.182.78:
    Packets: Sent = 4, Received = 4, Lost = 0 (0% loss),
Approximate round trip times in milli-seconds:
    Minimum = 10ms, Maximum = 14ms, Average = 12ms
```

Task 5 – Secure Input Handling in Web Forms

Prompt:

Generate a simple Flask web application with an HTML feedback form that accepts user input (name and message) and displays the submitted data on the webpage without performing any validation or sanitization.

Code:

```python
import re
from datetime import datetime

from flask import Flask, render_template, request

app = Flask(__name__)
feedback_store: list[dict[str, str]] = []
def validate_form(name: str, email: str, message: str) -> dict[str, str]:
    errors: dict[str, str] = {}

    if not name or len(name) > 50 or not re.fullmatch(r"[A-Za-z .'-]+", name):
        errors["name"] = "Name must be 1 to 50 characters and contain letters only."

    if not re.fullmatch(r"[^@\s]+@[^@\s]+\.[^@\s]+", email):
        errors["email"] = "Enter a valid email address."

    if not message or len(message) < 10 or len(message) > 500:
        errors["message"] = "Message must be between 10 and 500 characters."

    return errors


@app.get("/")
def form_page():
    return render_template("task5_feedback_form.html", errors={}, form_data={}, success=None)
@app.post("/submit")
def submit_feedback():
    name = request.form.get("name", "").strip()
    email = request.form.get("email", "").strip()
    message = request.form.get("message", "").strip()
    errors = validate_form(name, email, message)
    if errors:
        return render_template(
            "task5_feedback_form.html",
            errors=errors,
            form_data={"name": name, "email": email, "message": message},
            success=None,
        ), 400
    feedback_store.append(
        {
            "name": name,
            "email": email,
            "message": message,
            "created_at": datetime.utcnow().isoformat(),
        }
    )
    return render_template(
        "task5_feedback_form.html",
        errors={},
        form_data={},
        success="Feedback submitted successfully.",
    )
if __name__ == "__main__":
    app.run(debug=False)
```

```
(.venv) PS D:\AI Assisted coding> & "d:/AI Assisted coding/.venv/Scripts/python.exe" "d:/AI Assisted coding/task5_secure_form.py"
 * Serving Flask app 'task5_secure_form'
 * Debug mode: off
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
 * Running on http://127.0.0.1:5000
Press CTRL+C to quit
(.venv) PS D:\AI Assisted coding>
```

Observation:

This code implements a secure feedback form using Flask. It validates user input for name, email, and message fields to prevent malicious data. The form data is stored in memory with a timestamp. The use of regex ensures that only valid characters are accepted, and the length checks prevent excessively long inputs.